EduFinder

find your university.

A Web Application to Explore India's Top Universities,
Their Courses and Indicative Annual Fees

Project Report

Bachelor of Technology

Submitted by

Manthan

Sharda University

Academic Year 2025–26

Acknowledgement

I would like to express my sincere gratitude to my faculty and mentors at Sharda University for their guidance and encouragement throughout the development of this project. Their feedback helped shape both the idea and its execution.

I am also thankful to my peers for testing early versions of the application and offering honest, constructive suggestions that improved its usability. Finally, I acknowledge the National Institutional Ranking Framework (NIRF) and the open-source communities behind Flask and Supabase, whose data and tools made this work possible.

Manthan
B.Tech, Sharda University

Abstract

Choosing a university is a high-stakes decision that students in India routinely make with fragmented and inconsistent information. Rankings, course offerings and fee structures live in separate places and formats, making fair comparison difficult and time-consuming.

EduFinder is a web application that solves this by consolidating the top 100 NIRF-ranked universities — together with the courses they offer and indicative annual fees — into a single searchable, filterable and sortable interface. Users can search by university name, city, state or course; narrow results by state or by an annual-fee budget; sort by ranking, rating or fees; and open a detail page that visualises each course's fee for easy comparison.

The system is implemented with a Python-first, three-layer architecture: a Flask backend for logic, Jinja2 templates for server-rendered UI, and a Supabase (PostgreSQL) data model relating universities to their courses. The dataset spans 100 universities, 493 courses, 22 states and 61 cities. This report presents the problem, objectives, design, implementation, testing and future scope of the application, and reflects on its honest treatment of indicative fee data as a deliberate design choice.

Keywords: *university discovery*, *NIRF rankings*, *course fees*, *Flask*, *Jinja2*, *Supabase*, *PostgreSQL*, *web application*.

Table of Contents

Acknowledgement
Abstract

1. Introduction
2. Problem Statement & Objectives
3. Literature / Existing Systems
4. System Requirements
5. System Design & Architecture
6. Data & Methodology
7. Implementation
8. Features & User Interface
9. Testing
10. Limitations, Future Scope & Conclusion

1. Introduction

EduFinder is a web application designed to help students explore and compare universities in India. Every year, lakhs of students who finish school face the same difficult question — which university should they join, for which course, and at what cost. The information needed to answer this question exists, but it is scattered across ranking portals, individual university websites, prospectus PDFs, and third-party aggregators. Comparing options fairly is therefore slow and error-prone.

EduFinder addresses this by consolidating three key pieces of information — a university's national ranking, the courses it offers, and the indicative annual fee for each course — into a single, searchable interface. The application currently covers the top 100 NIRF-ranked universities in India, spanning 22 states and 61 cities, with 493 representative courses in total.

The project is built with a Python-first stack: a Flask backend performs all search, filtering and sorting logic, Jinja2 templates render the results as responsive cards, and the data layer is modelled on Supabase (PostgreSQL) with two related tables. This report describes the motivation, design, implementation and testing of the application, and outlines a roadmap for its future development.

2. Problem Statement & Objectives

2.1 Problem Statement

Selecting a university is one of the most consequential decisions a student makes, yet it is often made with incomplete and inconsistent data. The specific difficulties are:

- Scattered information — rankings, course lists and fees are published in different places and formats.
- Unclear fees — annual fees are hard to locate, vary by course and category, and are rarely presented in a comparable way.
- No easy comparison — students cannot quickly filter universities by state, budget or ranking in one view.
- Decision pressure — a high-stakes choice ends up being made under time pressure with fragmented data.

2.2 Objectives

1. To aggregate India's top 100 NIRF-ranked universities into a single dataset.
2. To allow students to search universities by name, city, state or course.
3. To provide filtering by state and by an annual-fee budget ceiling.
4. To allow sorting by NIRF rank, rating, or annual fee.
5. To present each university's courses with an at-a-glance visual fee comparison.
6. To keep the system simple, fast, and extensible to a live database.

3. Literature / Existing Systems

Several established portals provide information about Indian higher education. Understanding their strengths and gaps helped shape EduFinder's focus.

Existing system	Strength	Gap addressed by EduFinder
NIRF portal	Authoritative government rankings	No fees or course-level comparison
Aggregator sites	Large volume of listings	Cluttered, ad-heavy, inconsistent
University sites	Accurate, official data	One institution at a time; no compare

The common gap is the absence of a lightweight tool that brings ranking, courses and indicative fees together in one comparable, distraction-free view. EduFinder targets exactly this gap.

4. System Requirements

4.1 Software Requirements

- Python 3.10 or higher
- Flask (web framework)
- Jinja2 templating engine (bundled with Flask)
- Supabase / PostgreSQL for the production data layer
- A modern web browser (Chrome, Edge, Firefox)

4.2 Hardware Requirements

- Any computer capable of running Python (2 GB RAM or more)
- Approximately 50 MB of free disk space

4.3 Functional Requirements

- The system shall list universities as cards showing rank, rating, location and starting fee.
- The system shall support keyword search and state/budget filtering.
- The system shall sort results by rank, rating, or fee.
- The system shall display a detail page of courses and fees for each university.

5. System Design & Architecture

5.1 Architecture Overview

EduFinder follows a classic three-layer server-rendered architecture. The browser sends an HTTP request; the Flask backend reads the query parameters, filters and sorts the in-memory dataset in Python, and passes the result to a Jinja2 template; the template renders HTML which is returned to the browser. There is no client-side framework, which keeps the application small and easy to reason about.

5.2 Request Flow

1. User submits a search query or adjusts a filter on the home page.
2. Flask route reads parameters (query, state, max_fee, sort).
3. The dataset is filtered and sorted in Python.
4. Jinja2 renders the matching universities as cards.
5. Selecting a card opens its detail page with course-level fee bars.

5.3 Data Model

The data layer models two related tables. A university has many courses; each course belongs to one university through a foreign key. A single LEFT JOIN, aggregated per university, yields the JSON structure the application consumes.

Table	Key columns	Relationship
universities	id, name, location, state, rating, nirf_rank	One-to-many with courses
courses	id, university_id (FK), course_name, annual_fee	Belongs to one university

6. Data & Methodology

6.1 University Selection

The universities and their ranks are drawn from the National Institutional Ranking Framework (NIRF), published by the Ministry of Education. The top 10 reflect the NIRF 2025 ranking; ranks 11–100 follow the NIRF 2024 university list. Together these give a well-recognised, authoritative ordering of 100 institutions across the country.

6.2 Fee Estimation

Exact, official fees for every course at 100 universities are not available in any single consolidated source, and they change annually and by category. For a prototype, EduFinder therefore uses indicative annual fees derived from the type of each institution. Fees are grouped into representative bands, illustrated below (values in rupees per year):

Institution type	Example flagship course	Indicative fee / yr
Central / Government university	B.A. (Hons), B.Sc (Hons)	₹12,000
State university	B.Sc Computer Science	₹15,000
Government technical	B.Tech Computer Science	₹1,60,000
Private technical (deemed)	B.Tech Computer Science	₹2,95,000
Private premium	B.E. Computer Science	₹5,20,000
Private medical (deemed)	MBBS	₹16,50,000

Because these are indicative figures, the application clearly labels them as such on every detail page and advises users to confirm exact, current fees on each university's official website. This honesty is a deliberate design decision, not an oversight.

7. Implementation

7.1 Project Structure

The codebase is intentionally small and organised by responsibility:

- app.py — the Flask application: routes, and the search / filter / sort logic.
- data.py — the dataset of 100 universities and 493 courses.
- templates/index.html — the home page with the search bar, filters and card grid.
- templates/university.html — the per-university detail page with fee bars.
- static/ — the EduFinder logo and favicon (SVG).

7.2 Key Logic

The home route reads four query parameters — the search text, the selected state, a maximum-fee ceiling, and a sort key. It filters the dataset so that a university matches when the query appears in its name, city, state or any of its course names, its state matches the selected filter, and its cheapest course is within the fee ceiling. The filtered list is then sorted by the chosen key (NIRF rank, rating, or fee ascending/descending) and handed to the template.

A small helper formats fees for display, converting large numbers into a compact form (for example, ₹2,95,000 becomes '₹3.0L' and ₹40,000 becomes '₹40K'). The detail route looks up a single university by its id and computes the maximum course fee so that each course's fee bar can be drawn in proportion.

7.3 Front-End Rendering

Jinja2 templates receive the Python data directly and loop over it to produce HTML. University cards display initials in a coloured banner, the NIRF rank as a badge, and the location, rating, starting fee and course count. The

detail page renders each course as a labelled bar whose width is proportional to its fee, giving an immediate visual sense of relative cost.

8. Features & User Interface

The application presents two main screens — a discovery home page and a university detail page — tied together by a consistent navy-and-amber brand system.

8.1 Home Page

- A prominent search bar matching names, cities, states and courses.
- A state dropdown (22 states) and a fee slider to cap the budget.
- A sort control (NIRF rank, rating, fees low→high, fees high→low).
- A responsive grid of university cards, each linking to its detail page.

8.2 Detail Page

- A header showing the university's name, location, rating and NIRF rank.
- A list of courses, each with its indicative annual fee and a proportional bar.
- A clear notice that fees are indicative and should be verified officially.

8.3 Design System

EduFinder uses a deliberate two-colour identity: a deep navy (1A1F36) as the dominant colour and a warm amber (E8A020) as the accent, with a book-and-search-lens logo that visually communicates the app's purpose. Cards, badges and fee bars reuse these colours consistently across both screens.

9. Testing

The application was verified with a set of functional tests covering routing, search, filtering, sorting and edge cases. Representative test cases and their outcomes are summarised below.

#	Test case	Expected	Result
1	Open home page	100 cards shown	**Pass**
2	Search 'computer'	Only CS-offering unis	**Pass**
3	Filter state = Tamil Nadu	Only TN universities	**Pass**
4	Fee slider = ₹15,000	Only low-fee unis (38)	**Pass**
5	Sort by fees (low→high)	Cheapest first	**Pass**
6	Open detail /university/3	MAHE courses + fees	**Pass**
7	Invalid id /university/500	404 handled	**Pass**

All test cases passed, confirming that the core discovery workflow behaves as intended.

10. Limitations, Future Scope & Conclusion

10.1 Limitations

- Fees are indicative and grouped by institution type, not official per-course quotes.
- Only flagship courses are listed, not the complete catalogue of every degree.
- The dataset is a static snapshot; rankings and fees change over time.

10.2 Future Scope

- Integrate verified, category-wise fees with links to official sources.
- Add side-by-side comparison, shortlists and student accounts.
- Include student reviews, cut-offs and admission deadlines.
- Connect the live Supabase database and expose a public REST API.

10.3 Conclusion

EduFinder demonstrates how a focused, well-designed web application can simplify a genuinely difficult decision for students. By bringing rankings, courses and indicative fees together in one searchable, filterable and sortable interface, it removes much of the friction of comparing universities. The clean three-layer architecture — Flask, Jinja2 and a PostgreSQL data model — keeps the system easy to understand and ready to scale from a prototype into a full production service. With the planned enhancements, EduFinder can grow into a trusted companion for students navigating higher-education choices in India.

References

- National Institutional Ranking Framework (NIRF), Ministry of Education — nirfindia.org
- Flask documentation — flask.palletsprojects.com
- Supabase documentation — supabase.com/docs